Software Re-Engineering

Lecture 15

Reverse engineering III – Starting with IDA, IDA Displays I

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

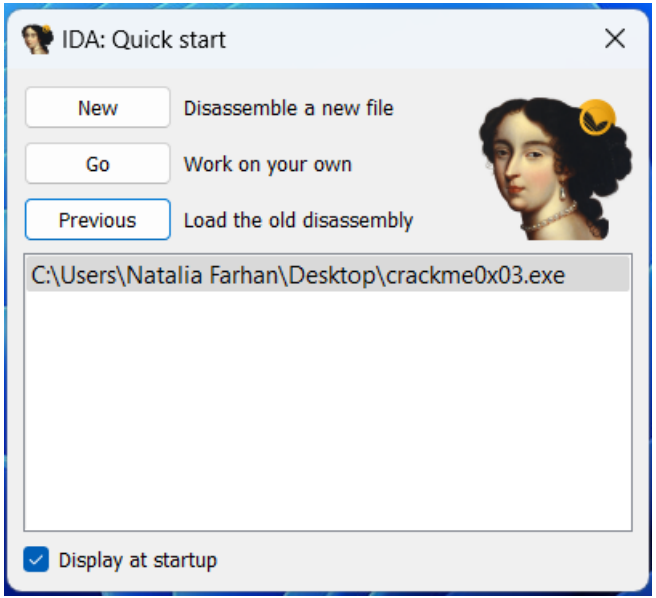
IDA Disassembler

Download IDA Free
https://hex-rays.com/ida-free/#download

Crack
https://github.com/Maijin/radare2-workshop-2015/blob/master/IOLI-crackme/bin-win32/crackme0x03.exe

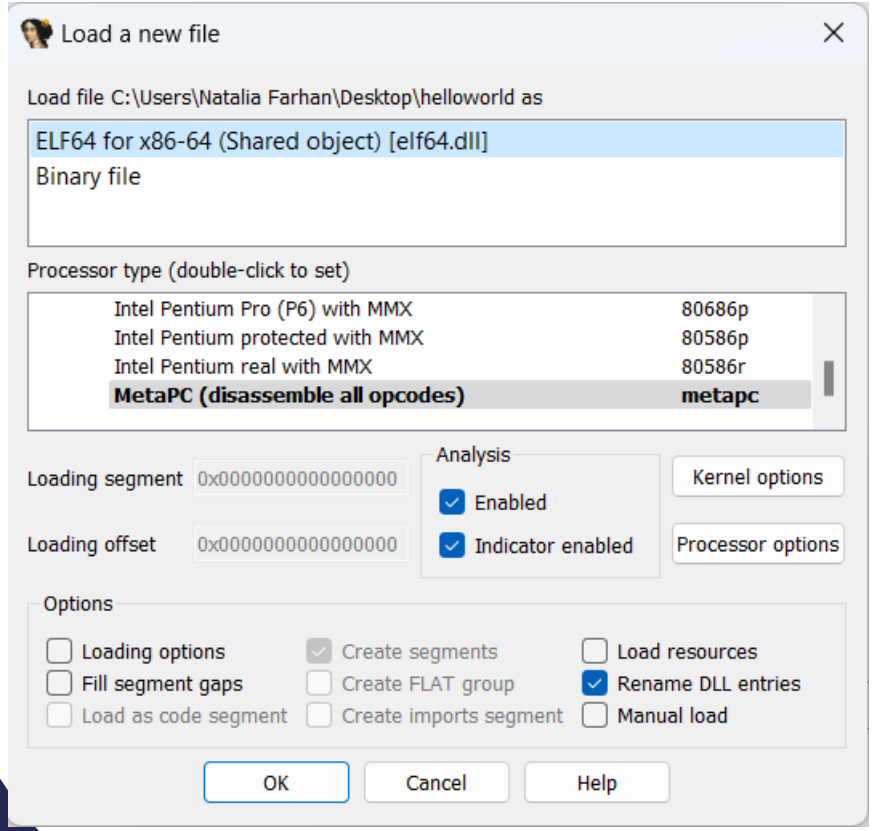
Readings

Chapter 4: GETTING STARTED WITH IDA, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle



Get the binary and add it through **new** file

https://github.com/BIackMage/ReverseEngineeringPractice/blob/main/binaries/helloworld



ELF extensible and linkable format is a standrad binary format

Database files

Upon opening a new file to analyze with IDA Pro, it analyzes the whole executable file and creates an.idb database archive. The .idb archive contains four files:

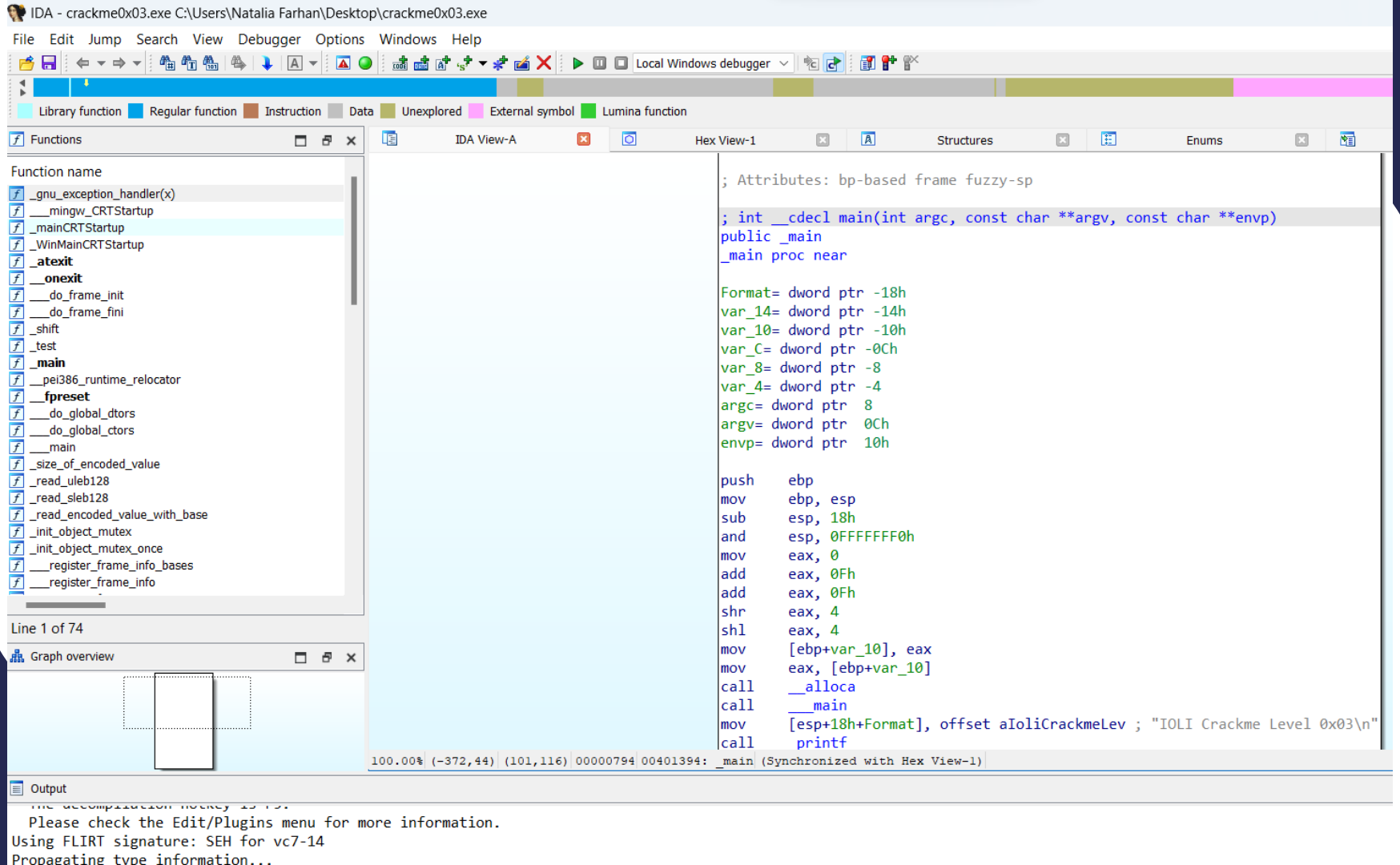
name.id0 - contains contents of B-tree style database,
name.id1 - contains flags that describe each program byte,
name.nam - contains index information related to named program locations,
name.til - contains information about local type definitions

- Once the .idb database has been created for a specific executable, IDA won't need to analyze the program again when we load it later.
- Moreover, IDA doesn't even require the executable anymore; we can now work with just the .idb file.
- This is a useful feature that can be used to pass around .idb files to other researchers without the malicious executable.
- Therefore, IDA can analyze the executable without the actual executable, and with only the database archive file.

Anytime we're trying to close the currently open.idb database (the currently analyzed executable), IDA asks us if we would like to save changes to the disk. We can choose from the following options:

- Don't pack database: flush changes to .id0, id1, nam and til databases and don't create .idb file.
- Pack database (Store): archives the .id0, id1, nam and til into the .idb archive. Note that the .idb of the previous session is overwritten.
- Pack database (Deflate): the same as the previous option, except the database files are compressed in the .idb archive.
- Collect garbage: deletes any unused memory pages from the database. This can be useful if we want to create a smaller database .idb file.
- Don't save the database: we can pick this option if we don't want to save the changes that we have made.

- During that time, IDA will also load and parse the actual code instructions of the executable file into the assembly instructions of the selected processor module.
- Those assembly instructions are then also showed to the user for analysis.
- But IDA doesn't stop there; it can also scan the generated assembly instructions to figure out additional information about the executable, like the compiler which was used to compile the executable, the function's arguments, the function's local variables, etc.

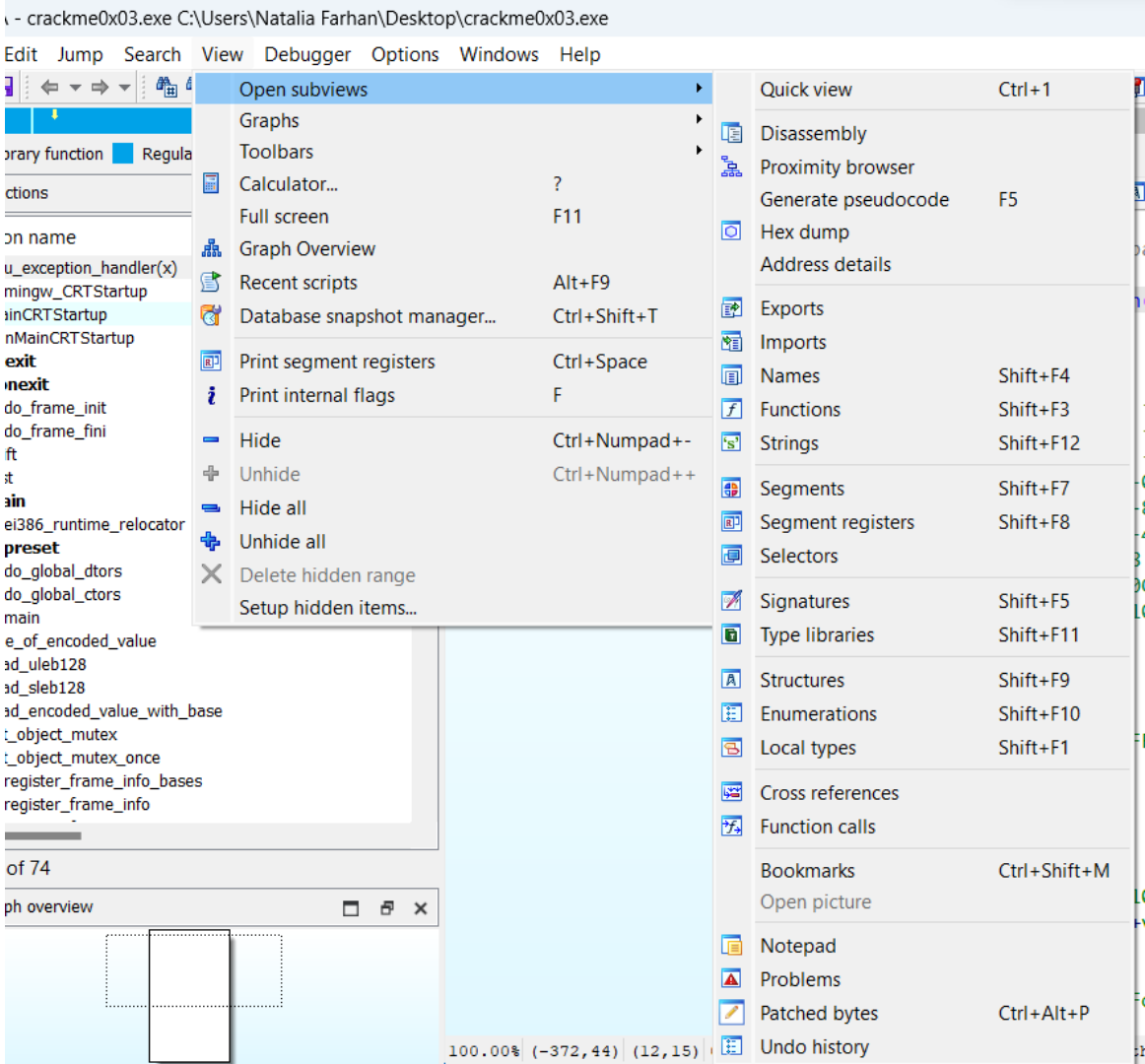

IDA - crackme0x03.exe C:\Users\Natalia Farhan\Desktop\crackme0x03.exe
File Edit Jump Search View Debugger Options Windows Help
Local Windows debugger
Library function Regular function Instruction Data Unexplored External symbol Lumina function
Functions
Function name
_gnu_exception_handler(x)
___mingw_CRTStartup
_mainCRTStartup
_WinMainCRTStartup
_atexit
__onexit
___do_frame_init
___do_frame_fini
_shift
_test
_main
__pei386_runtime_relocator
__fpreset
___do_global_dtors
___do_global_ctors
___main
_size_of_encoded_value
_read_uleb128
_read_sleb128
_read_encoded_value_with_base
_init_object_mutex
_init_object_mutex_once
___register_frame_info_bases
___register_frame_info
Line 1 of 74
Graph overview
IDA View-A
Hex View-1
Structures
Enums
; Attributes: bp-based frame fuzzy-sp
; int __cdecl main(int argc, const char **argv, const char **envp)
public _main
_main proc near
Format= dword ptr -18h
var_14= dword ptr -14h
var_10= dword ptr -10h
var_C= dword ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4
argc= dword ptr 8
argv= dword ptr 0Ch
envp= dword ptr 10h
push ebp
mov ebp, esp
sub esp, 18h
and esp, 0FFFFFFF0h
mov eax, 0
add eax, 0Fh
add eax, 0Fh
shr eax, 4
shl eax, 4
mov [ebp+var_10], eax
mov eax, [ebp+var_10]
call __alloca
call ___main
mov [esp+18h+Format], offset aIoliCrackmeLev ; "IOLI Crackme Level 0x03\n"
call _printf
100.00% (-372,44) (101,116) 00000794 00401394: _main (Synchronized with Hex View-1)
Output
Please check the Edit/Plugins menu for more information.
Using FLIRT signature: SEH for vc7-14
Propagating type information...

Readings

Chapter 5: IDA DATA DISPLAYS, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle

The Principal IDA Displays

- To open a specific data view, we can go to View - Open Subviews and choose the appropriate view we would like to show.
- We can also switch back to the default view by clicking on Windows - Reset desktop.



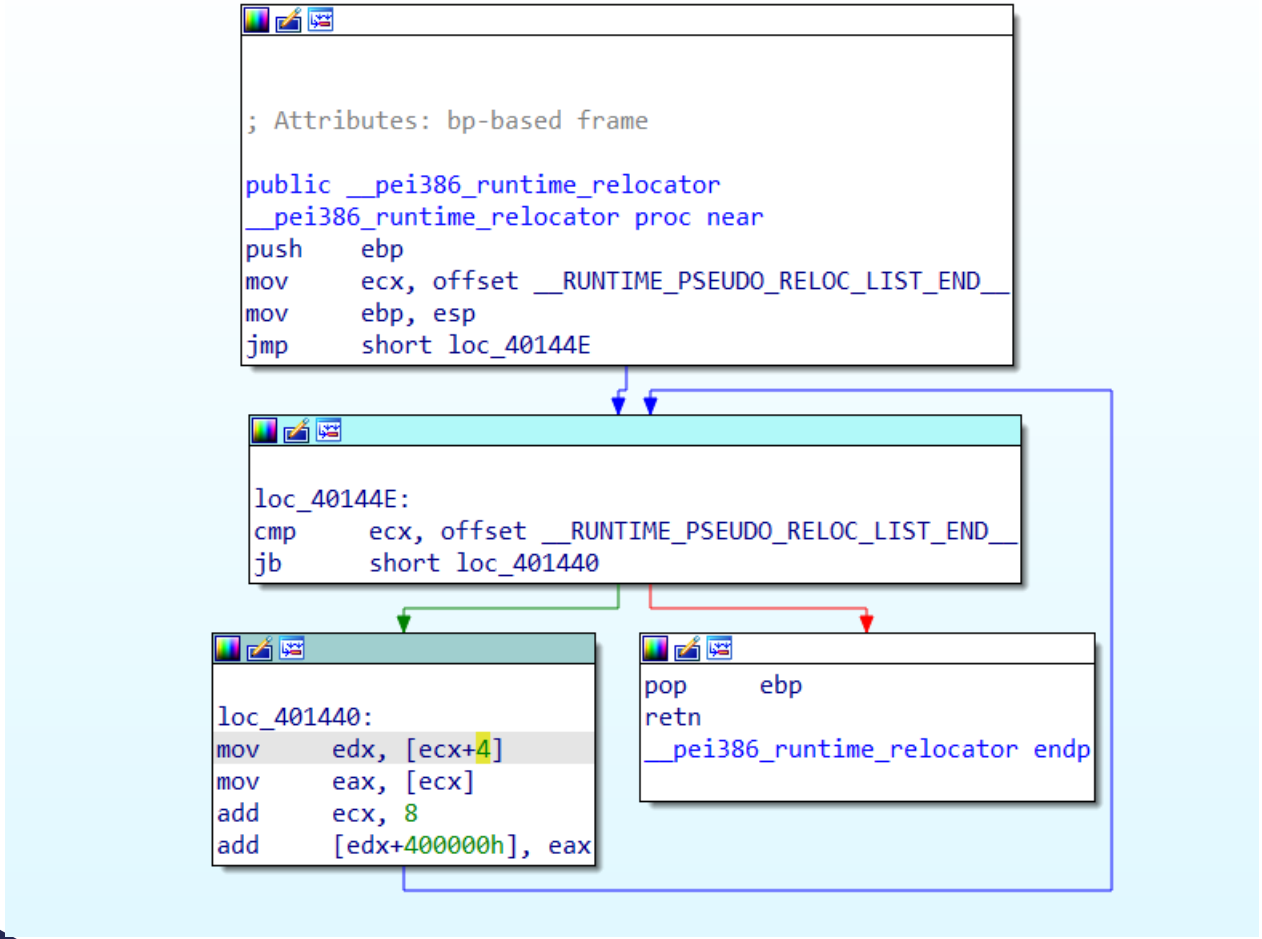
The graph view can be used if we want to quickly figure out the execution flow of the current function and the listing view can be used when we want to see the actual assembly instructions.

On the graph overview, the program is broken down into logical blocks, where each block is presenting a jump target (as defined in the assembly code). From the graph overview we can also see the logic the program uses while executing.

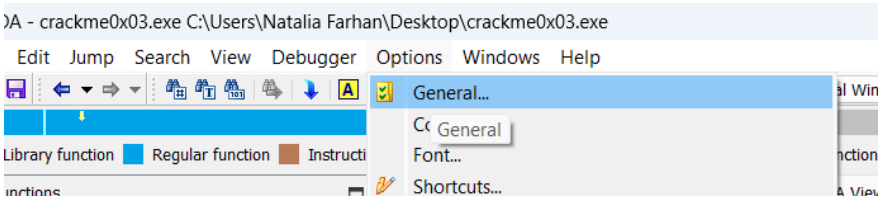
- A basic block is a maximal sequence of instructions that executes, without branching, from beginning to end.
- Each basic block therefore has a single entry point (the first instruction in the block) and a single exit point (the last instruction in the block).
- The first instruction in a basic block is often the target of a branching instruction, while the last instruction in a basic block is often a branch instruction

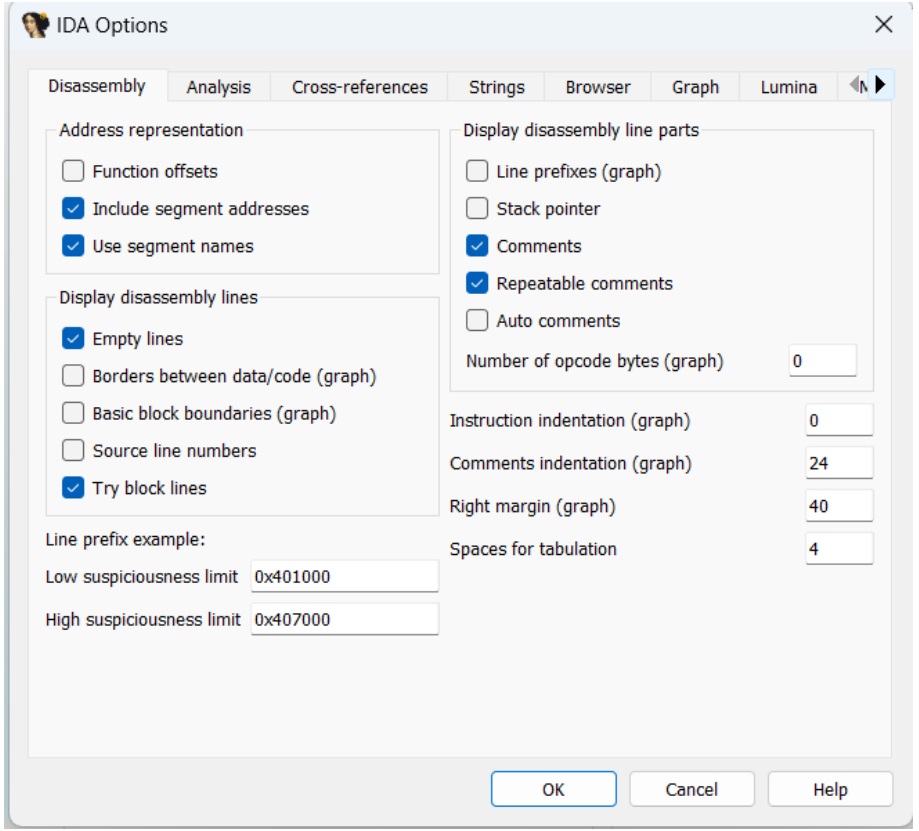
Graph views are somewhat reminiscent of program flowcharts in that a function is broken up into **basic blocks** so you can visualize the function's control flow from one block to another.

- Onscreen, you’ll notice IDA uses different colored arrows to distinguish various types of flows2 between the blocks of a function.
- `Basic blocks that terminate with a conditional jump generate two possible flows depending on the condition being tested: the Yes edge arrow (yes, the branch is taken) is green by default, and the No edge arrow (no, the branch is not taken) is red by default.
- Basic blocks that terminate with only one potential successor block utilize a Normal edge (blue by default) to point to the next block to be executed.


; Attributes: bp-based frame
public __pei386_runtime_relocator
__pei386_runtime_relocator proc near
push ebp
mov ecx, offset __RUNTIME_PSEUDO_RELOC_LIST_END__
mov ebp, esp
jmp short loc_40144E
loc_40144E:
cmp ecx, offset __RUNTIME_PSEUDO_RELOC_LIST_END__
jb short loc_401440
loc_401440:
mov edx, [ecx+4]
mov eax, [ecx]
add ecx, 8
add [edx+400000h], eax
pop ebp
retn
__pei386_runtime_relocator endp

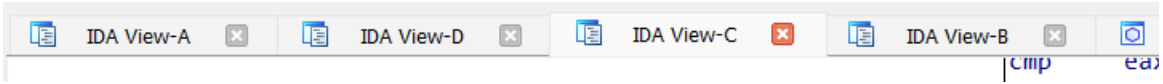
- When using graph view, it may seem as if less information is available to you about each line of the disassembly.
- The reason for this is that IDA chooses to hide many of the more traditional pieces of information about each disassembled line (such as virtual address information) in order to minimize the amount of space required to display each basic block.
- You can choose to display additional information with each disassembly line by choosing among the available disassembly line parts accessible via the Disassembly tab from Options->General. For example, to add virtual addresses to each disassembly line, we enable line prefixes


)A - crackme0x03.exe C:\Users\Natalia Farhan\Desktop\crackme0x03.exe
Edit Jump Search View Debugger Options Windows Help
General...
General
Font...
Shortcuts...
Library function
Regular function
Instructi


IDA Options
Disassembly
Analysis
Cross-references
Strings
Browser
Graph
Lumina
Address representation
Function offsets
Include segment addresses
Use segment names
Display disassembly lines
Empty lines
Borders between data/code (graph)
Basic block boundaries (graph)
Source line numbers
Try block lines
Line prefix example:
Low suspiciousness limit 0x401000
High suspiciousness limit 0x407000
Display disassembly line parts
Line prefixes (graph)
Stack pointer
Comments
Repeatable comments
Auto comments
Number of opcode bytes (graph) 0
Instruction indentation (graph) 0
Comments indentation (graph) 24
Right margin (graph) 40
Spaces for tabulation 4
OK
Cancel
Help

Creating additional disassembly windows

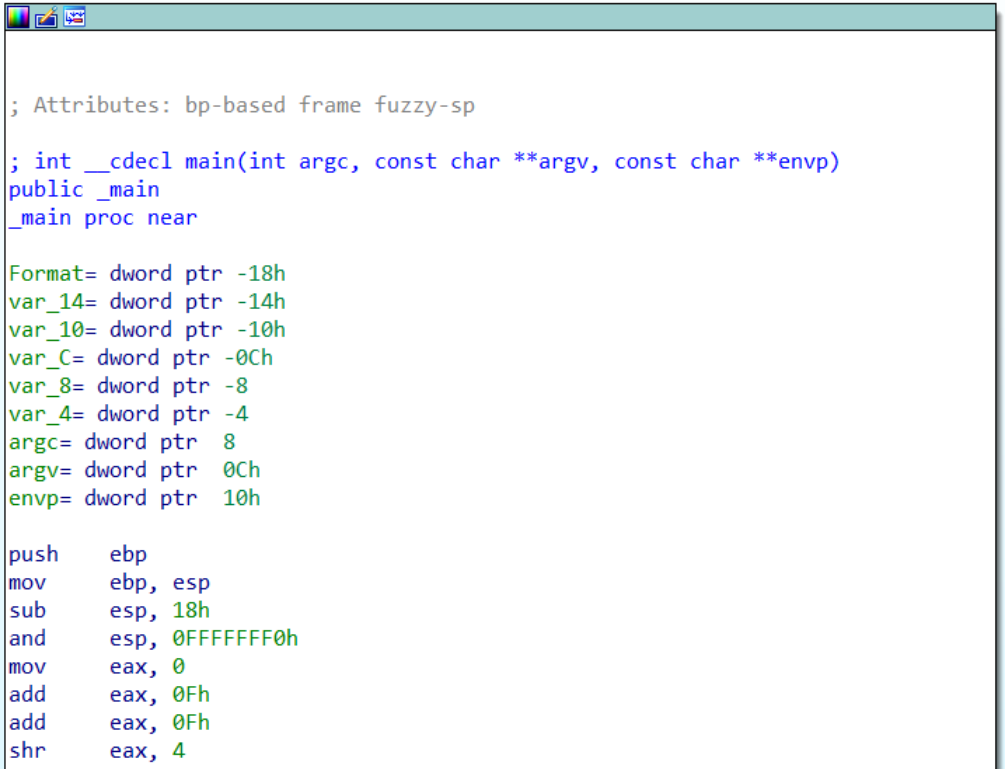
- If you ever find yourself wanting to view graphs of two functions simultaneously, all you need to do is open another disassembly window using Views->Open Subviews->Disassembly.
- The first disassembly window opened is titled IDA View-A. Subsequent disassembly windows are titled IDA View-B, IDA View-C, and so on.
- Each disassembly is independent of the other, and it is perfectly acceptable to view a graph in one window while viewing a text listing in another or to view three different graphs in three different windows



IDA View-A | IDA View-D | IDA View-C | IDA View-B | Hex View-1 | Structures

IDA Text View

- The text-oriented disassembly window is the traditional display used for viewing and manipulating IDA-generated disassemblies.
- The text display presents the entire disassembly listing of a program (as opposed to a single function at a time in graph mode) and provides the only means for viewing the data regions of a binary.



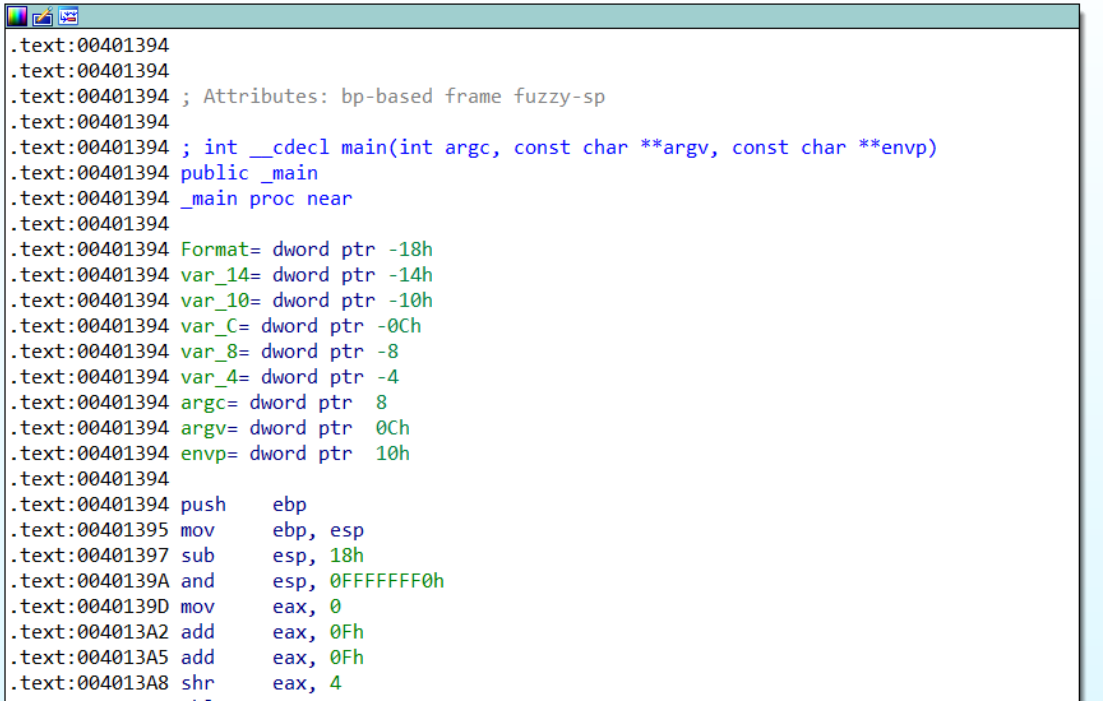
```
; Attributes: bp-based frame fuzzy-sp

; int __cdecl main(int argc, const char **argv, const char **envp)
public _main
_main proc near

Format= dword ptr -18h
var_14= dword ptr -14h
var_10= dword ptr -10h
var_C= dword ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4
argc= dword ptr  8
argv= dword ptr  0Ch
envp= dword ptr  10h

push    ebp
mov     ebp, esp
sub     esp, 18h
and     esp, 0FFFFFFF0h
mov     eax, 0
add     eax, 0Fh
add     eax, 0Fh
shr     eax, 4
```

- The disassembly is presented in linear fashion, with virtual addresses displayed by default.
- Virtual addresses are typically displayed in a [SECTION NAME]:[VIRTUAL ADDRESS] format such as .text:004011C1.



```
.text:00401394
.text:00401394
.text:00401394 ; Attributes: bp-based frame fuzzy-sp
.text:00401394
.text:00401394 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:00401394 public _main
.text:00401394 _main proc near
.text:00401394
.text:00401394 Format= dword ptr -18h
.text:00401394 var_14= dword ptr -14h
.text:00401394 var_10= dword ptr -10h
.text:00401394 var_C= dword ptr -0Ch
.text:00401394 var_8= dword ptr -8
.text:00401394 var_4= dword ptr -4
.text:00401394 argc= dword ptr  8
.text:00401394 argv= dword ptr  0Ch
.text:00401394 envp= dword ptr  10h
.text:00401394
.text:00401394 push    ebp
.text:00401395 mov     ebp, esp
.text:00401397 sub     esp, 18h
.text:0040139A and     esp, 0FFFFFFF0h
.text:0040139D mov     eax, 0
.text:004013A2 add     eax, 0Fh
.text:004013A5 add     eax, 0Fh
.text:004013A8 shr     eax, 4
```

The declarations (also present in graph view) represent IDA's best estimate concerning the layout of the function's stack frame

```
.text:00401394 Format= dword ptr -18h
.text:00401394 var_14= dword ptr -14h
.text:00401394 var_10= dword ptr -10h
.text:00401394 var_C= dword ptr -0Ch
.text:00401394 var_8= dword ptr -8
.text:00401394 var_4= dword ptr -4
.text:00401394 argc= dword ptr  8
.text:00401394 argv= dword ptr  0Ch
.text:00401394 envp= dword ptr  10h
```

IDA uses those markers to ease the reading of assembly instructions relating to the stack.

Once IDA detects an offset would point into the function's stack as either an internal variable or an argument, it'll assign a name to the specific stack offsets each parameter and/or argument points to from the base of the stack.

It then calculates all offsets to the same position regardless of stack growth/shrinking and will use the stack offset names where-ever they're possible.

The syntax is the following:

<argument/variable name> = <size> ptr <offset from stack base>

```
.text:00401394 Format= dword ptr -18h
.text:00401394 var_14= dword ptr -14h
.text:00401394 var_10= dword ptr -10h
```

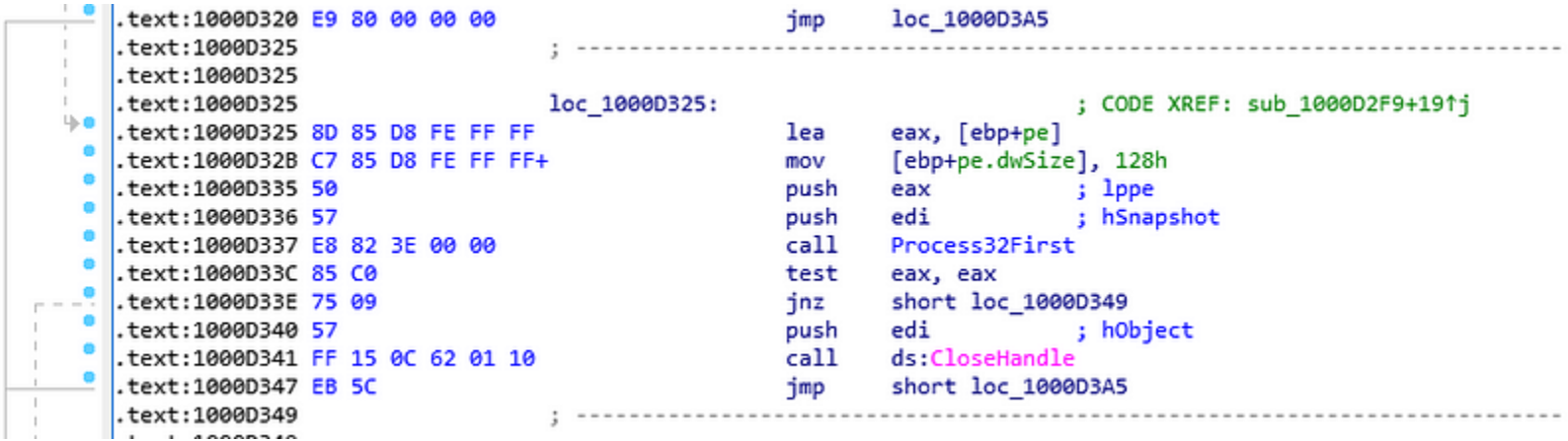
Cross-references, or more commonly referred to as xrefs, are used to identify references (usage/call or declaration) of a particular function, string, variable, etc. Example — if you've identified a particular function being called inside another subroutine, you might be interested in knowing other calls to the same function.

Cross-references can be categorized as follows:

- Code cross-references
- Data cross-references

Code Cross-references

Code cross-references are used to identify relationships between function calls, declarations, or jumps.



```
.text:1000D320 E9 80 00 00 00                  jmp     loc_1000D3A5
.text:1000D325                 ; ---------------------------------------------------------------------------
.text:1000D325
.text:1000D325                 loc_1000D325:                           ; CODE XREF: sub_1000D2F9+19↑j
.text:1000D325 8D 85 D8 FE FF FF               lea     eax, [ebp+pe]
.text:1000D32B C7 85 D8 FE FF FF+              mov     [ebp+pe.dwSize], 128h
.text:1000D335 50                              push    eax             ; lppe
.text:1000D336 57                              push    edi             ; hSnapshot
.text:1000D337 E8 82 3E 00 00                  call    Process32First
.text:1000D33C 85 C0                           test    eax, eax
.text:1000D33E 75 09                           jnz     short loc_1000D349
.text:1000D340 57                              push    edi             ; hObject
.text:1000D341 FF 15 0C 62 01 10               call    ds:CloseHandle
.text:1000D347 EB 5C                           jmp     short loc_1000D3A5
.text:1000D349                 ; ---------------------------------------------------------------------------
```

Here, we can see that **loc_1000D325** has a code cross-reference to **sub_1000D2F9+19**. It suggests that this location is accessed by the **function (sub_*)** at the address **1000D2F9** and **an offset of 0x19** into the function. Next, the arrow shows us that the code is at a lower address than the reference and the ‘j’ suggests it’s a jump.

- A code cross-reference is used to indicate that an instruction transfers or may transfer control to another instruction.
- The manner in which instructions transfer control is referred to as a flow within IDA.
- IDA distinguishes among three basic flow types: ordinary, jump, and call.
- Jump and call flows are further divided according to whether the target address is a near or far address.

```
int read_it; //integer variable read in main
int write_it; //integer variable written 3 times in main
int ref_it; //integer variable whose address is taken in main
void callflow() {} //function called twice from main

int main() {
   int *p = &ref_it;    //results in an "offset" style data reference
   *p = read_it;        //results in a "read" style data reference
   write_it = *p;       //results in a "write" style data reference
   callflow();          //results in a "call" style code reference
   if (read_it == 3) {  //results in "jump" style code reference
      write_it = 2;     //results in a "write" style data reference
   }
   else {               //results in an "jump" style code reference
      write_it = 1;     //results in a "write" style data reference
   }
   callflow();          //results in an "call" style code reference
 }
```

```
.text:00401010 _main           proc near
.text:00401010
.text:00401010 p               = dword ptr -4
.text:00401010
.text:00401010                 push    ebp
.text:00401011                 mov     ebp, esp
.text:00401013                 push    ecx
.text:00401014                 ❾mov    [ebp+p], offset ref_it
.text:0040101B                 mov     eax, [ebp+p]
.text:0040101E                 ❼mov    ecx, read_it
.text:00401024                 mov     [eax], ecx
.text:00401026                 mov     edx, [ebp+p]
.text:00401029                 mov     eax, [edx]
.text:0040102B                 ❽mov    write_it, eax
.text:00401030                 ❸call   callflow
.text:00401035                 ❼cmp    read_it, 3
.text:0040103C                 jnz     short loc_40104A
.text:0040103E                 ❽mov    write_it, 2
.text:00401048                 ❶jmp    short loc_401054
❺ .text:0040104A ; ---------------------------------------------------------------
.text:0040104A
.text:0040104A loc_40104A:                             ❻; CODE XREF: _main+2C↑j
.text:0040104A                 ❽mov    write_it, 1
.text:00401054
.text:00401054 loc_401054:                             ❻; CODE XREF: _main+38↑j
.text:00401054                 ❸call   callflow
.text:00401059                 xor     eax, eax
.text:0040105B                 mov     esp, ebp
.text:0040105D                 pop     ebp
.text:0040105E                 ❷retn
.text:0040105E _main           endp
```

disassembly of the callflow function

```
.text:00401000 callflow        proc near               ; CODE XREF: _main+20↓p
.text:00401000                                          ; _main:loc_401054↓p
.text:00401000                 push    ebp
.text:00401001                 mov     ebp, esp
.text:00401003                 pop     ebp
.text:00401004                 retn
.text:00401004 callflow        endp
```

two cross-references are displayed at the address of callflow to indicate that the function is called twice

1010 (main_) + 20 (offset) = **1030**

An ordinary flow is the simplest flow type, and it represents sequential flow from one instruction to another.

```
int read_it; //integer variable read in main
int write_it; //integer variable written 3 times in main
int ref_it; //integer variable whose address is taken in main
void callflow() {} //function called twice from main

int main() {
   int *p = &ref_it;    //results in an "offset" style data reference
   *p = read_it;        //results in a "read" style data reference
   write_it = *p;       //results in a "write" style data reference
   callflow();          //results in a "call" style code reference
   if (read_it == 3) {  //results in "jump" style code reference
      write_it = 2;     //results in a "write" style data reference
   }
   else {               //results in an "jump" style code reference
      write_it = 1;     //results in a "write" style data reference
   }
   callflow();          //results in an "call" style code reference
}
```

Instructions used to invoke functions are assigned a call flow, indicating transfer of control to the target function.

Cross-references resulting from function calls are distinguished through use of the p suffix (think P for Procedure).

```
int read_it; //integer variable read in main
int write_it; //integer variable written 3 times in main
int ref_it; //integer variable whose address is taken in main
void callflow() {} //function called twice from main

int main() {
   int *p = &ref_it;    //results in an "offset" style data reference
   *p = read_it;        //results in a "read" style data reference
   write_it = *p;       //results in a "write" style data reference
   callflow();          //results in a "call" style code reference
   if (read_it == 3) {  //results in "jump" style code reference
      write_it = 2;     //results in a "write" style data reference
   }
   else {               //results in an "jump" style code reference
      write_it = 1;     //results in a "write" style data reference
   }
   callflow();          //results in an "call" style code reference
 }
```

A jump flow is assigned to each unconditional and conditional branch instruction.

Conditional branches are also assigned ordinary flows to account for control flow when the branch is not taken.

Unconditional branches have no associated ordinary flow because the branch is always taken in such cases.

Jump cross-references are distinguished by the use of a j suffix (think J for Jump).

Data Cross-references

The three most commonly encountered types of data cross-references are used to indicate

1. when a location is being read,
2. when a location is being written, and
3. when the address of a location is being taken.

```
.data:0040B720 read_it      dd ?                    ; DATA XREF: _main+E↑r
.data:0040B720                                      ; _main+25↑r
.data:0040B724 write_it     dd ?                    ; DATA XREF: _main+1B↑w
.data:0040B724                                     ⑩; _main+2E↑w ...
.data:0040B728 ref_it       db    ? ;               ; DATA XREF: _main+4↑o
.data:0040B729              db    ? ;
.data:0040B72A              db    ? ;
.data:0040B72B              db    ? ;
```

Much like code cross-references, data cross-references can help us keep track of data being accessed or written throughout the binary. For example, you might find an IP address being referenced in a few functions; using xrefs, you can identify all those functions easily.

```
xdoors_d:10094E64 43 6F 70 79 20 27+aCopySToSFailed   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Failed',0
xdoors_d:10094E64 25 73 27 20 54 6F+                                        ; DATA XREF: sub_1000D3D0:loc_1000D501↑o
xdoors_d:10094E64 20 27 25 73 27 20+                                        ; sub_1000DF22:loc_1000E59D↑o ...
xdoors_d:10094E7D 00 00 00                          align 10h
xdoors_d:10094E80                   ; char aCopySToSSucces[]
xdoors_d:10094E80 43 6F 70 79 20 27+aCopySToSSucces   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Successfully',0
xdoors_d:10094E80 25 73 27 20 54 6F+                                        ; DATA XREF: sub_1000D3D0+12A↑o
xdoors_d:10094E80 20 27 25 73 27 20+                                        ; sub_1000DF22+52F↑o ...
xdoors_d:10094E9F 00                                align 10h
xdoors_d:10094EA0                   ; char aThePidOfProces_1[]
xdoors_d:10094EA0 0D 0A 54 68 65 20+aThePidOfProces_1 db 0Dh,0Ah            ; DATA XREF: sub_1000D5B0+264↑o
xdoors_d:10094EA0 50 49 44 20 4F 66+                db 'The PID Of Process ',27h,'%s',27h,' Not Found',0Dh,0Ah,0
xdoors_d:10094EC6 00 00                             align 4
xdoors_d:10094EC8                   ; char aInjectSToProce_2[]
```

Here, we can see several data strings being referenced in functions. For example, the **aThePidofProces_1** is being referenced in the function, **sub_1000D5B0** and at on offset of 0x264. The direction is upwards again suggesting the function is at a lower address than string's address.

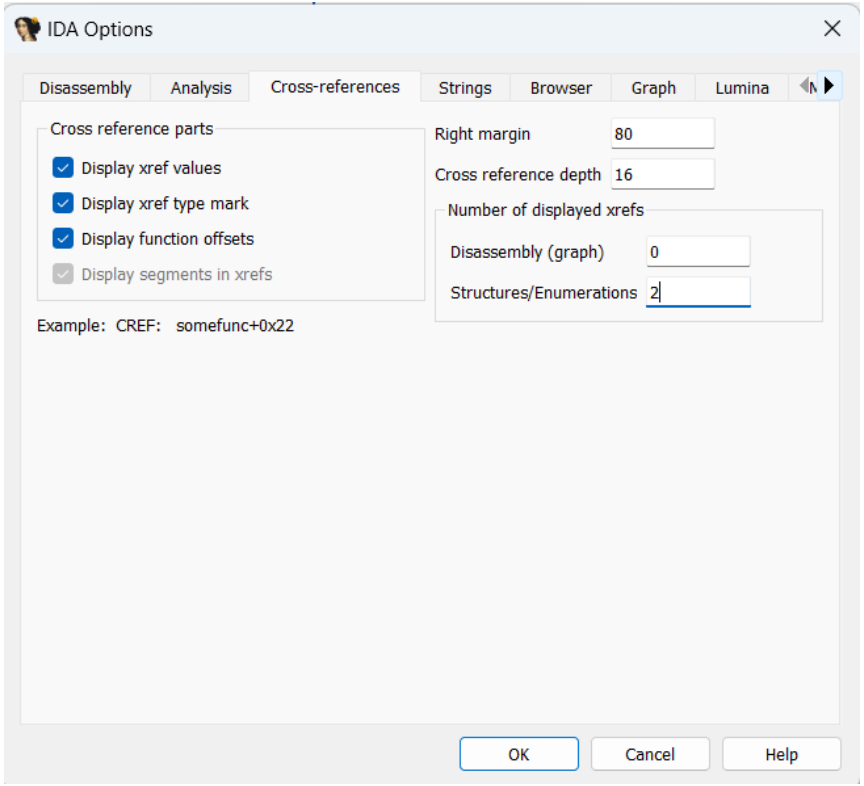
```
xdoors_d:10094E64 43 6F 70 79 20 27+aCopySToSFailed   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Failed',0
xdoors_d:10094E64 25 73 27 20 54 6F+                                        ; DATA XREF: sub_1000D3D0:loc_1000D501↑o
xdoors_d:10094E64 20 27 25 73 27 20+                                        ; sub_1000DF22:loc_1000E59D↑o ...
xdoors_d:10094E7D 00 00 00                          align 10h
xdoors_d:10094E80                   ; char aCopySToSSucces[]
xdoors_d:10094E80 43 6F 70 79 20 27+aCopySToSSucces db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Successfully',0
xdoors_d:10094E80 25 73 27 20 54 6F+                                        ; DATA XREF: sub_1000D3D0+12A↑o
xdoors_d:10094E80 20 27 25 73 27 20+                                        ; sub_1000DF22+52F↑o ...
xdoors_d:10094E9F 00                                align 10h
xdoors_d:10094EA0                   ; char aThePidOfProces_1[]
xdoors_d:10094EA0 0D 0A 54 68 65 20+aThePidOfProces_1 db 0Dh,0Ah            ; DATA XREF: sub_1000D5B0+264↑o
xdoors_d:10094EA0 50 49 44 20 4F 66+                db 'The PID Of Process ',27h,'%s',27h,' Not Found',0Dh,0Ah,0
xdoors_d:10094EC6 00 00                             align 4
xdoors_d:10094EC8                   ; char aInjectSToProce_2[]
```

For the DATA XREF to aCopySToSSucces you can see two cross-references being listed.

```
int read_it; //integer variable read in main
int write_it; //integer variable written 3 times in main
int ref_it; //integer variable whose address is taken in main
void callflow() {} //function called twice from main

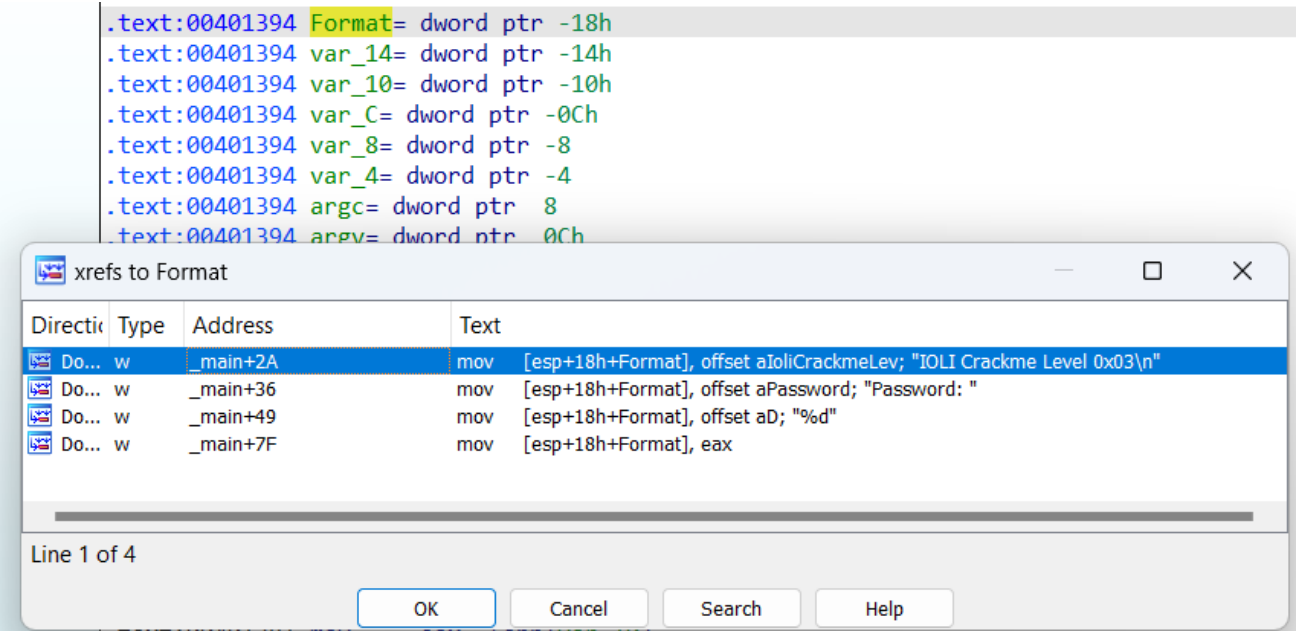
int main() {
   int *p = &ref_it;    //results in an "offset" style data reference
   *p = read_it;        //results in a "read" style data reference
   write_it = *p;       //results in a "write" style data reference
   callflow();          //results in a "call" style code reference
   if (read_it == 3) {  //results in "jump" style code reference
      write_it = 2;     //results in a "write" style data reference
   }
   else {               //results in an "jump" style code reference
      write_it = 1;     //results in a "write" style data reference
   }
   callflow();          //results in an "call" style code reference
 }
```

- IDA restricts listing multiple references by default. You can change this setting and show as many references as you want. It is under ‘Options’ — ‘General’ — ‘Cross-references’.



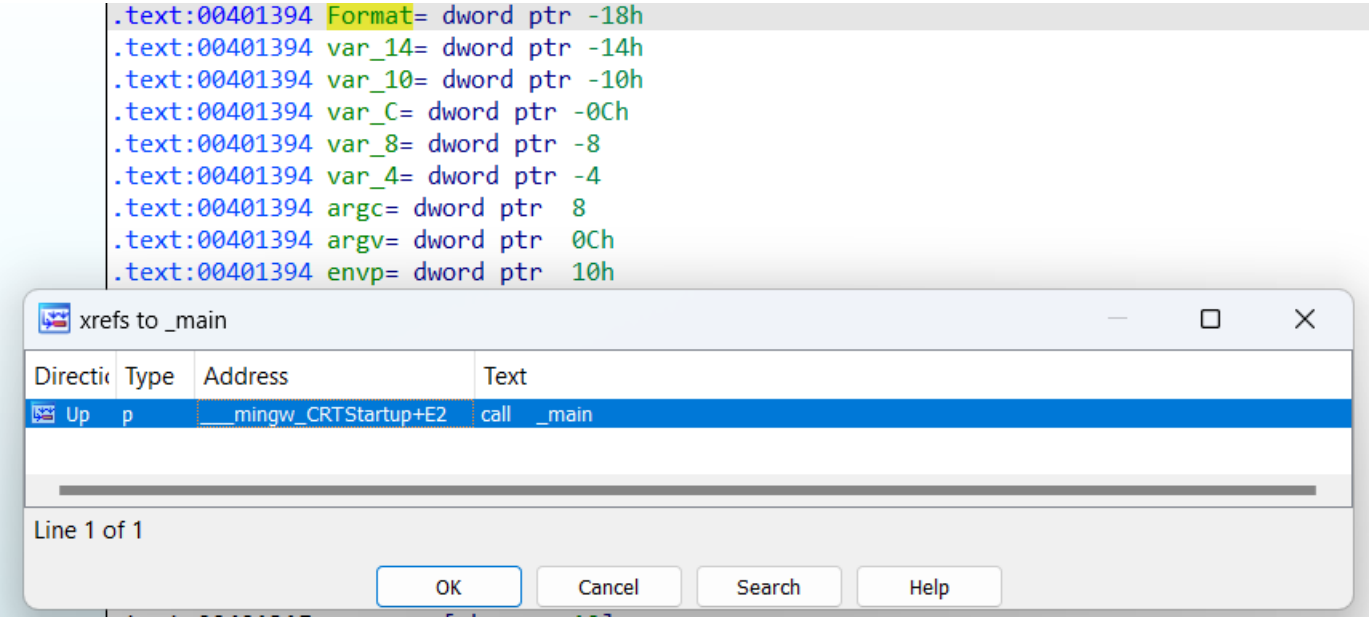
Listing Cross-references

- you can simply do so by selecting the label and pressing ‘X’ to list all cross-references to that particular label (or identifier).



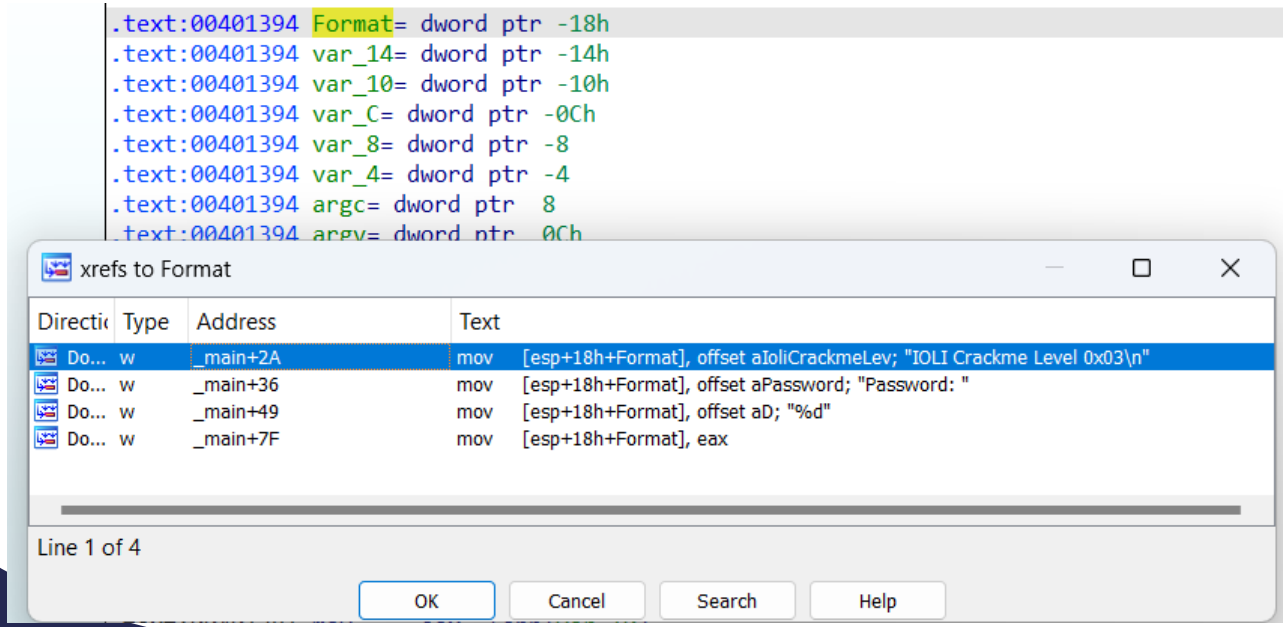
Listing Cross-references

- ‘CTRL+X’ shows us cross-references to the address (cursor’s selection does not matter as it is worked with addresses not identifiers).



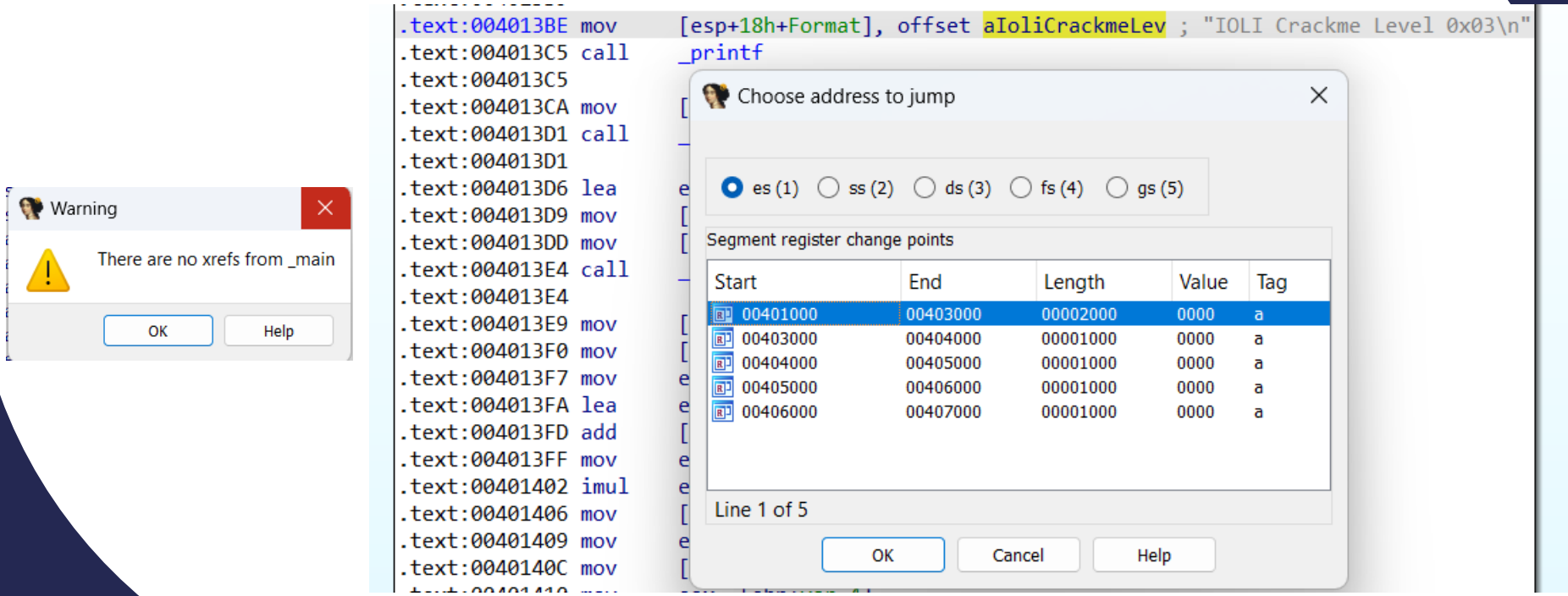
Here, we can find the following information:

- **Direction:** Whether the target reference is at a higher (Down) or lower (Up) address
- **Type:** Type of the cross-reference, using the type suffixes discussed previously
- **Address/Text:** the source address of the cross-reference, and the corresponding disassembled text at the source address



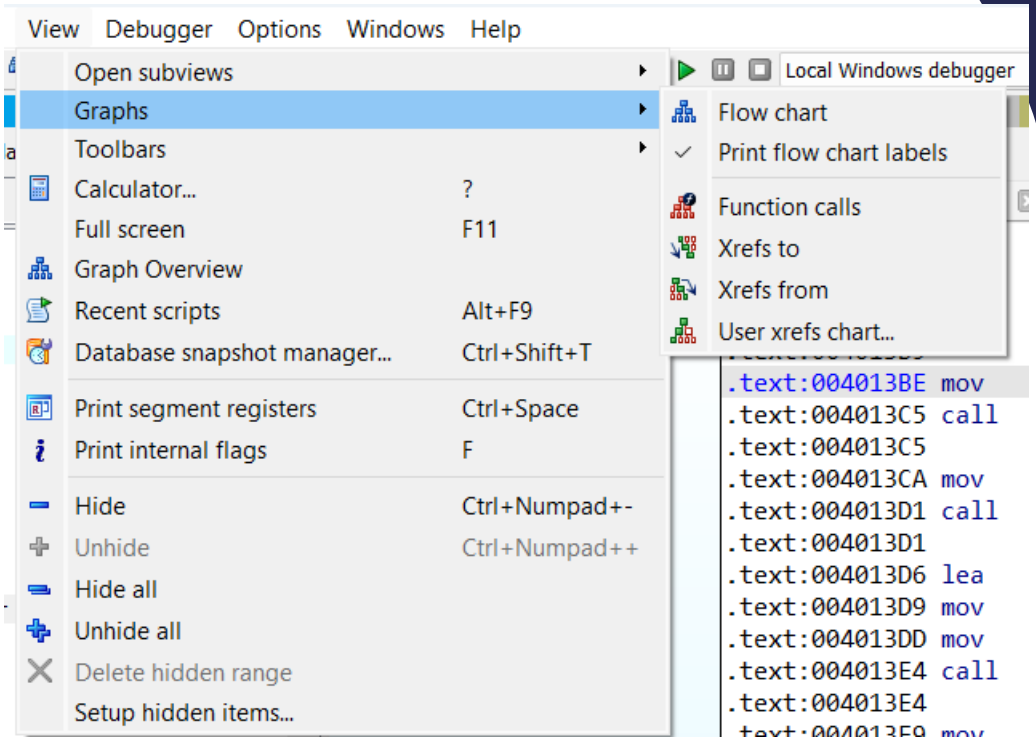
Listing Cross-references

- ‘CTRL+J’ shows us cross-references from the address



We can also view the information in a graphical view.
Select ‘View’, Go into ‘Graphs’ and select either of the three xref charts available.
You can:

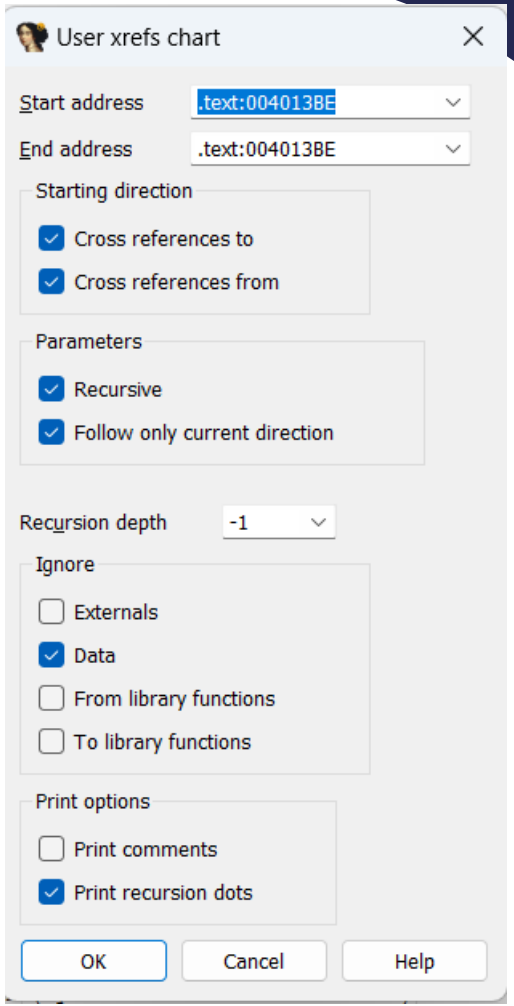
- View xrefs from the identifier (where the reference ‘goes to’)
- View xrefs to the identifier (where the reference ‘comes from’)
- Use a custom xref chart with several customization options (this is particularly helpful in larger binaries and to filter out unhelpful identifiers)



This command displays a user-defined graph of xrefs from/to the current address/range of selected addresses in the program.

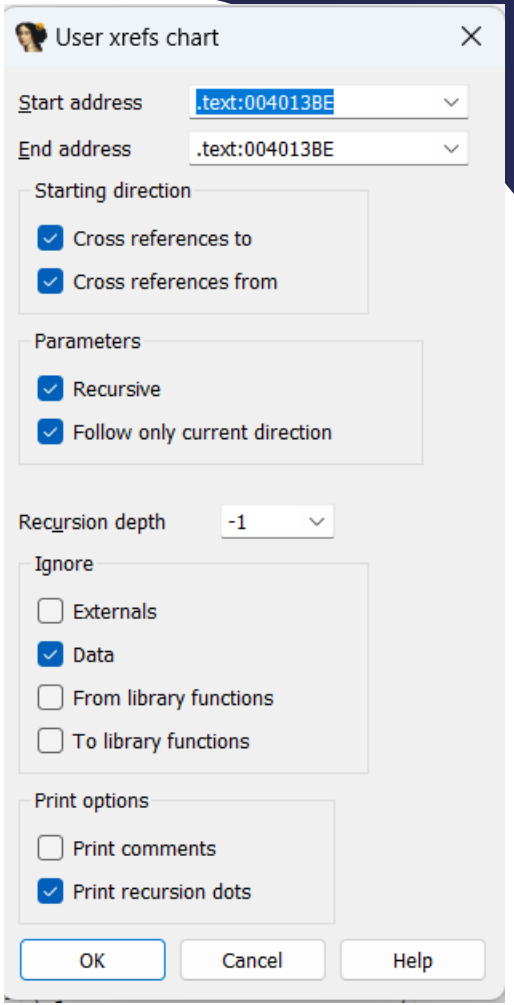
The direction of the xrefs to analyze can be chosen. Options allow you to decide whether to search for cross-references from the selected symbol, to the selected symbol, or both.

If all other options are left at their default settings, restricting the starting direction to **Cross references to** results in an Xrefs To–style graph, while restricting direction to **Cross references from** generates an Xrefs From–style graph



Parameters

- The Recursive option enables recursive descent (Xrefs From) or ascent (Xrefs To) from the selected symbols.
- **Follow only current direction** forces any recursion to occur in only one direction. In other words, if this option is selected, and node B is discovered to be reachable from node A, the recursive descent into B adds additional nodes that can be reached only from node B.
- If you choose to deselect **Follow only current direction**, then when both starting directions are selected, each new node added to the graph is recursed in both the to and from directions.

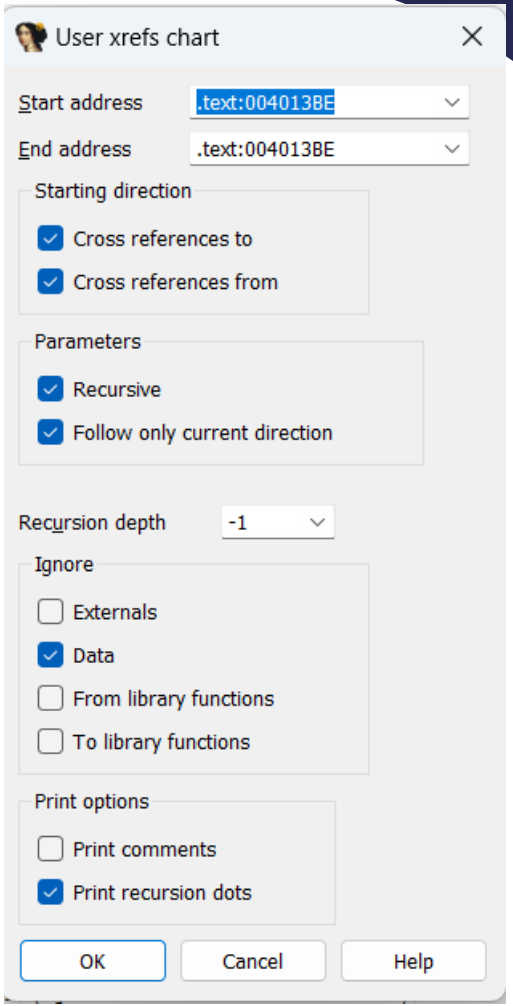


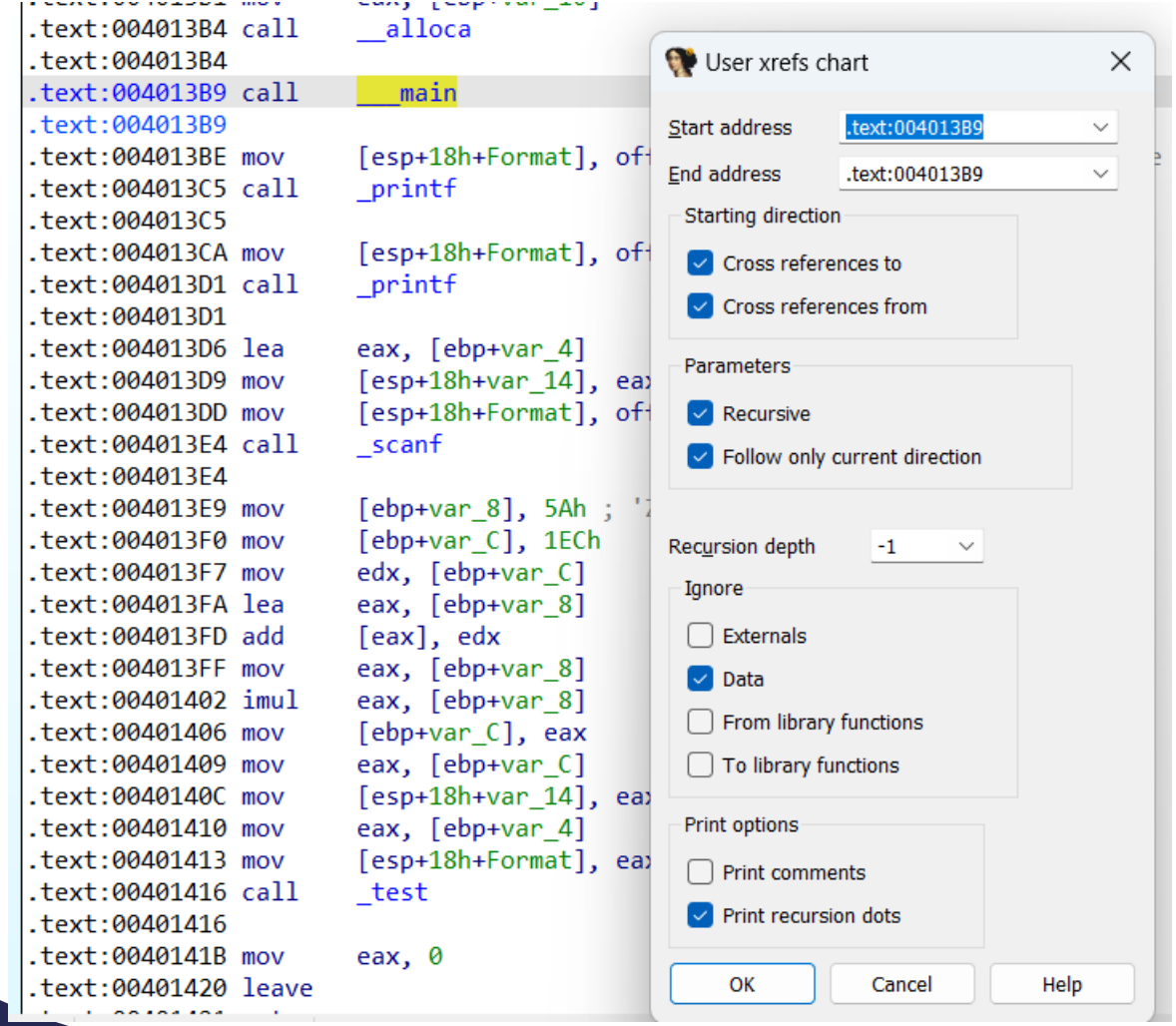
Only External, data xrefs, xrefs from library functions and to library functions can possibly be ignored.

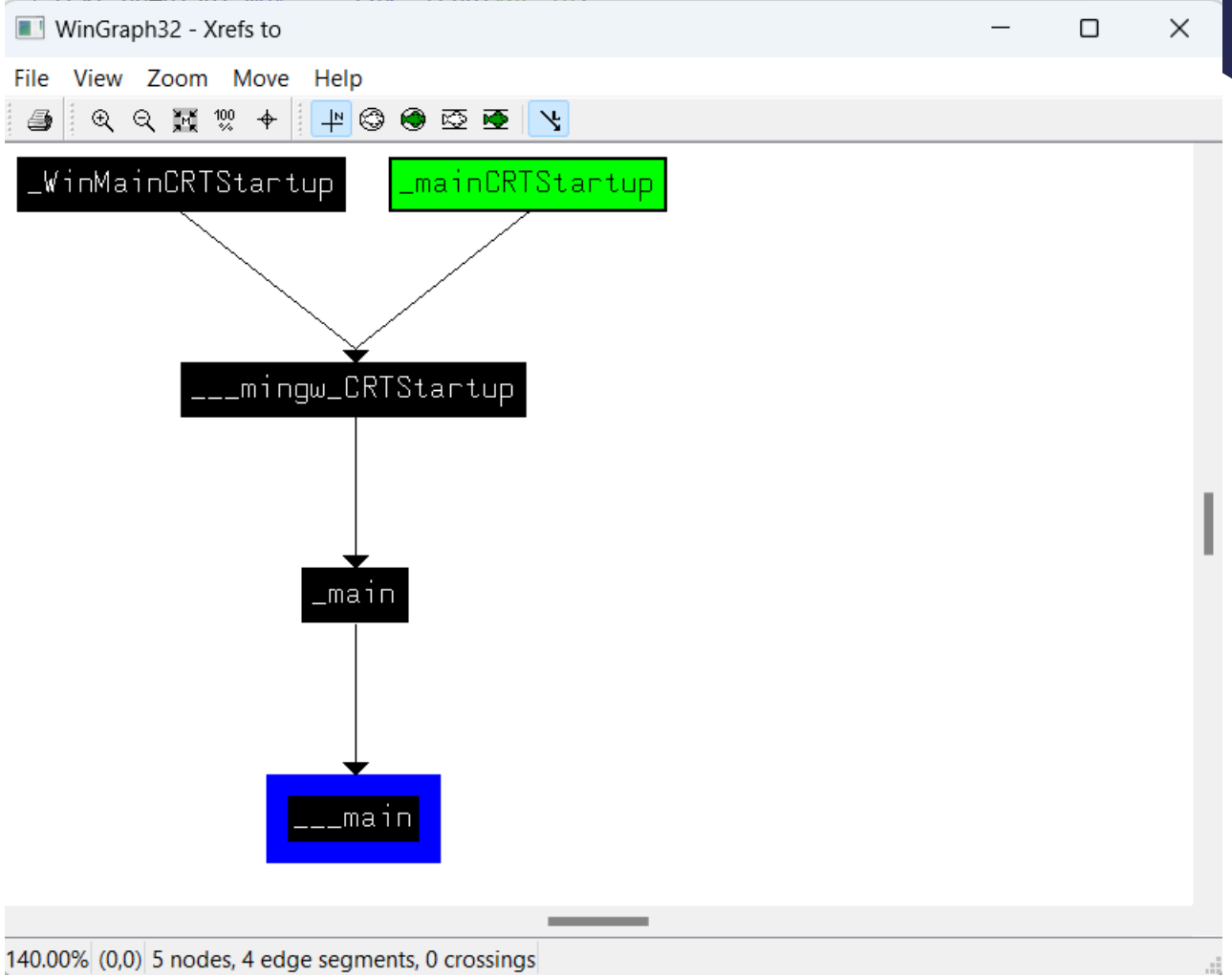
A recursion depth can be specified. If 'Print recursion dots' is checked, and a function has others xrefs outside of the range defined by the 'recursion depth' setting, small nodes containing dots are printed.

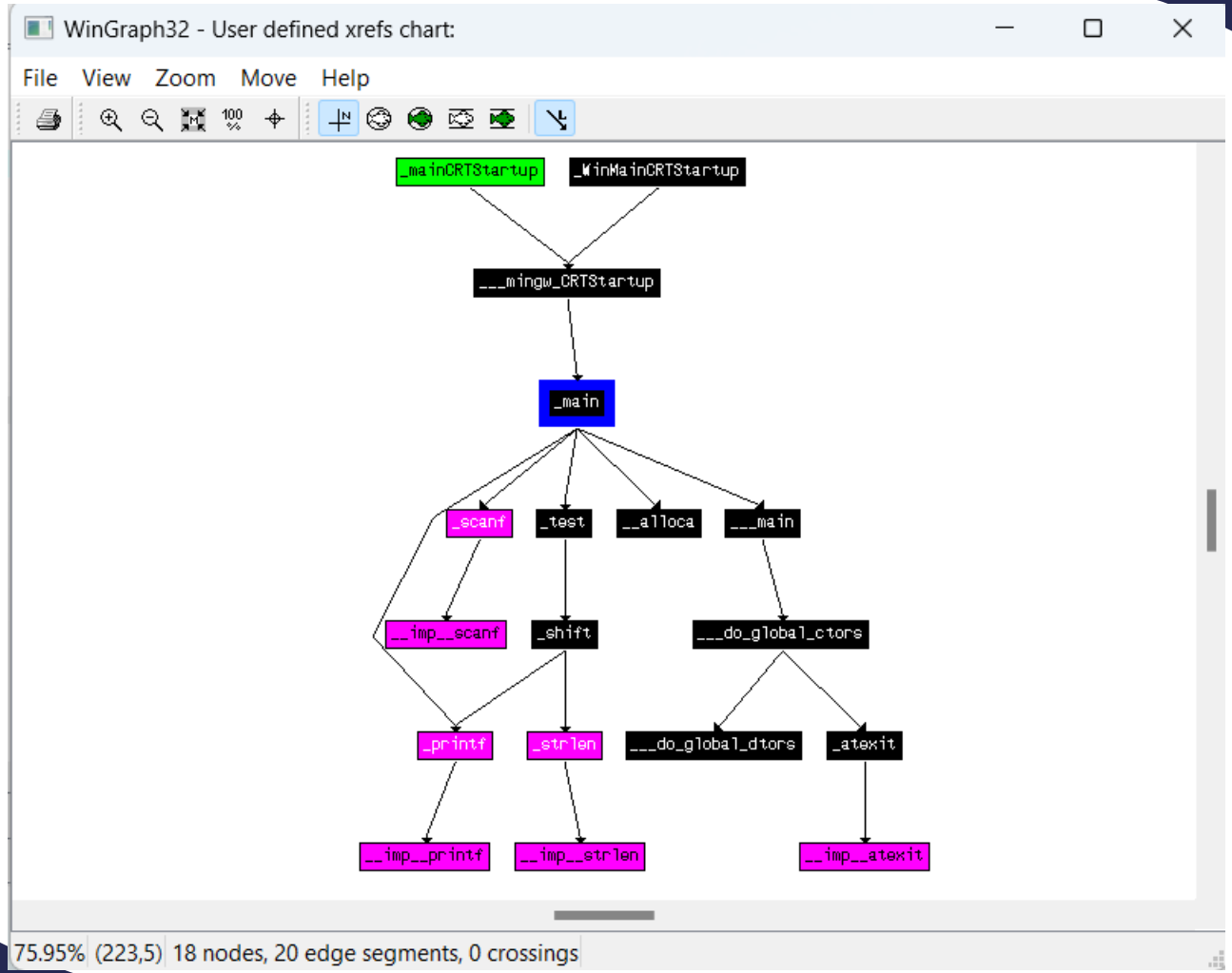
The 'Print comments' flag causes the generated function node to also contain the function comment.

The addresses are represented as nodes. The xrefs are represented as edges between nodes. The used colors are the same as in IDA.

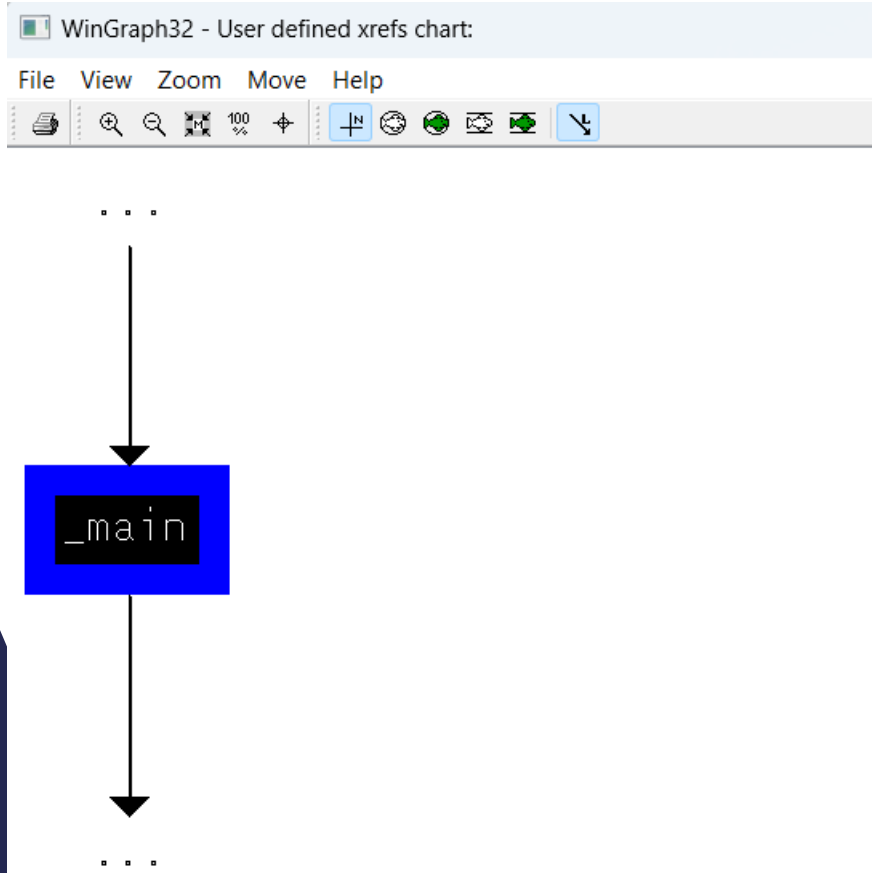



.text:004013B4 call __alloca
.text:004013B4
.text:004013B9 call __main
.text:004013B9
.text:004013BE mov [esp+18h+Format], of
.text:004013C5 call _printf
.text:004013C5
.text:004013CA mov [esp+18h+Format], of
.text:004013D1 call _printf
.text:004013D1
.text:004013D6 lea eax, [ebp+var_4]
.text:004013D9 mov [esp+18h+var_14], ea
.text:004013DD mov [esp+18h+Format], of
.text:004013E4 call _scanf
.text:004013E4
.text:004013E9 mov [ebp+var_8], 5Ah ; '
.text:004013F0 mov [ebp+var_C], 1ECh
.text:004013F7 mov edx, [ebp+var_C]
.text:004013FA lea eax, [ebp+var_8]
.text:004013FD add [eax], edx
.text:004013FF mov eax, [ebp+var_8]
.text:00401402 imul eax, [ebp+var_8]
.text:00401406 mov [ebp+var_C], eax
.text:00401409 mov eax, [ebp+var_C]
.text:0040140C mov [esp+18h+var_14], ea
.text:00401410 mov eax, [ebp+var_4]
.text:00401413 mov [esp+18h+Format], ea
.text:00401416 call _test
.text:00401416
.text:0040141B mov eax, 0
.text:00401420 leave
User xrefs chart
Start address .text:004013B9
End address .text:004013B9
Starting direction
Cross references to
Cross references from
Parameters
Recursive
Follow only current direction
Recursion depth -1
Ignore
Externals
Data
From library functions
To library functions
Print options
Print comments
Print recursion dots
OK
Cancel
Help

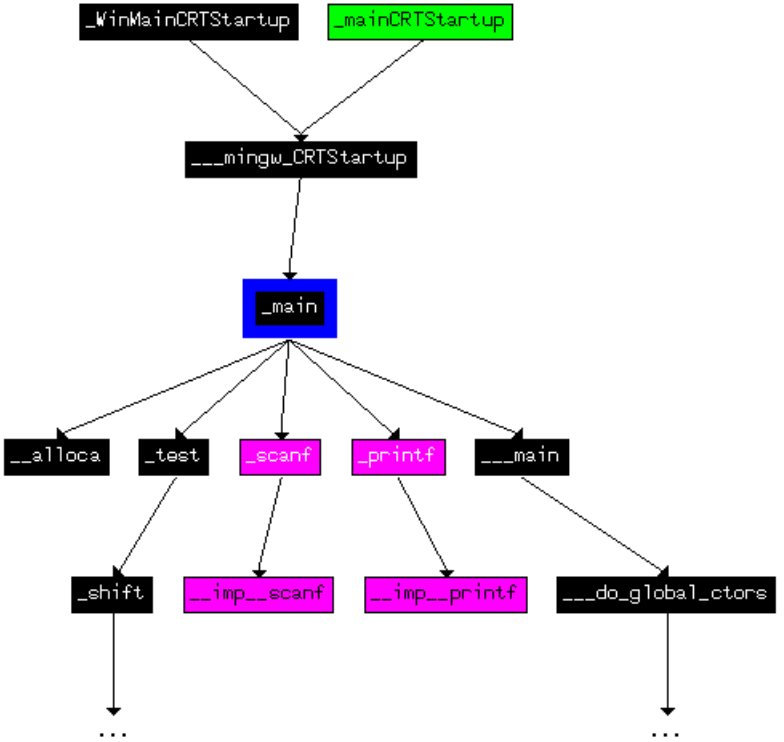

WinGraph32 - Xrefs to
File
View
Zoom
Move
Help
_WinMainCRTStartup
_mainCRTStartup
___mingw_CRTStartup
_main
___main
140.00%
(0,0)
5 nodes, 4 edge segments, 0 crossings


WinGraph32 - User defined xrefs chart:
File View Zoom Move Help
_mainCRTStartup
_WinMainCRTStartup
___mingw_CRTStartup
_main
_scanf
_test
__alloca
___main
__imp__scanf
_shift
___do_global_ctors
_printf
_strlen
___do_global_dtors
_atexit
__imp__printf
__imp__strlen
__imp__atexit
75.95% (223,5) 18 nodes, 20 edge segments, 0 crossings

Recursion depth : 0



Recursion depth : 2



Types of Cross-references

13 types of cross-references. A few of these cross-reference types are listed below:

O: Offset — Address is taken by the identifier selected by the cursor (there's no read or write operation here)

R: Read access — Data is being read from the address

W: Write access — Data is being written at the address

J: Far (Inter-segment) jump — Code being jumped to is in a different code segment than the current segment

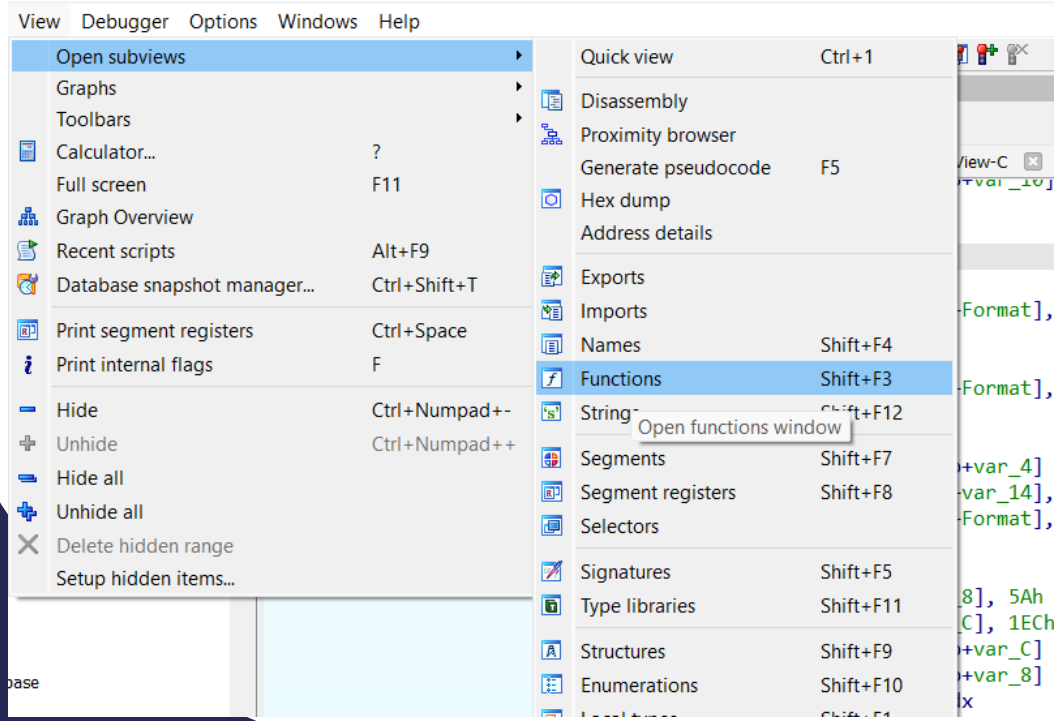
j: Near (Intra-segment) jump — Code being jumped to is in the same code segment as the identifier

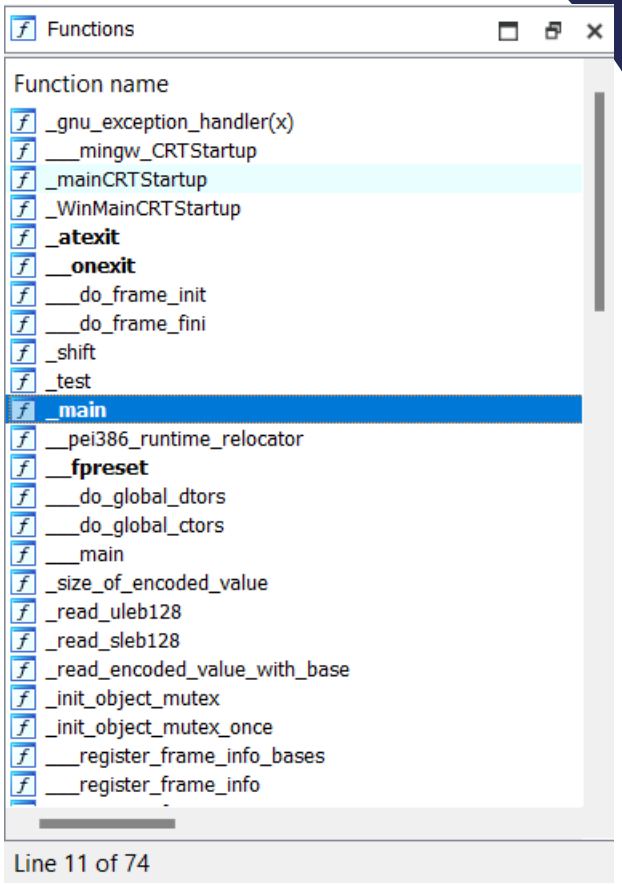
P: Far (Inter-segment) call — Call is in a different code segment

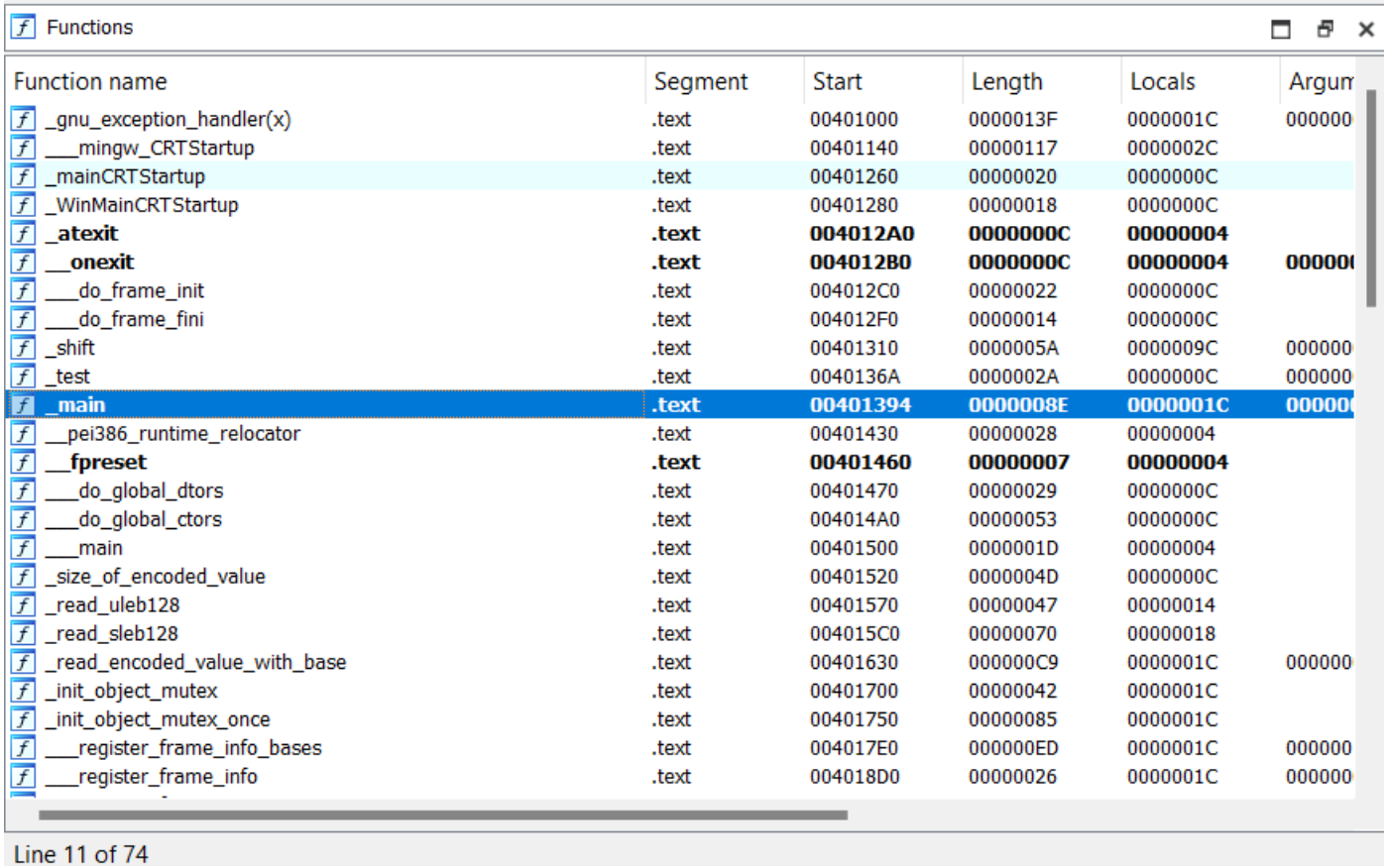
p: Near (Intra-segment) call — Call is in the same code segment

The Functions Window

The Functions window is used to list every function that IDA has recognized in the database





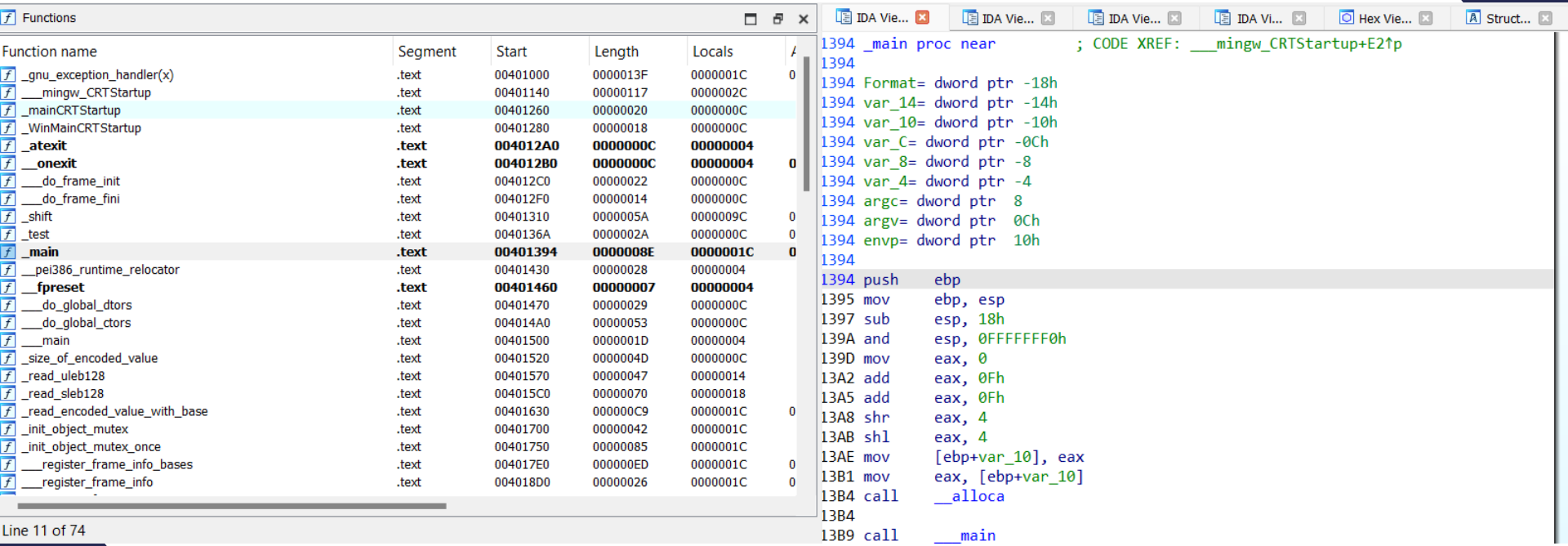

Functions
Function name
Segment
Start
Length
Locals
Argun
_gnu_exception_handler(x) .text 00401000 0000013F 0000001C 000000
___mingw_CRTStartup .text 00401140 00000117 0000002C
_mainCRTStartup .text 00401260 00000020 0000000C
_WinMainCRTStartup .text 00401280 00000018 0000000C
_atexit .text 004012A0 0000000C 00000004
__onexit .text 004012B0 0000000C 00000004 000000
___do_frame_init .text 004012C0 00000022 0000000C
___do_frame_fini .text 004012F0 00000014 0000000C
_shift .text 00401310 0000005A 0000009C 000000
_test .text 0040136A 0000002A 0000000C 000000
_main .text 00401394 0000008E 0000001C 000000
__pei386_runtime_relocator .text 00401430 00000028 00000004
__fpreset .text 00401460 00000007 00000004
___do_global_dtors .text 00401470 00000029 0000000C
___do_global_ctors .text 004014A0 00000053 0000000C
___main .text 00401500 0000001D 00000004
_size_of_encoded_value .text 00401520 0000004D 0000000C
_read_uleb128 .text 00401570 00000047 00000014
_read_sleb128 .text 004015C0 00000070 00000018
_read_encoded_value_with_base .text 00401630 000000C9 0000001C 000000
_init_object_mutex .text 00401700 00000042 0000001C
_init_object_mutex_once .text 00401750 00000085 0000001C
___register_frame_info_bases .text 004017E0 000000ED 0000001C 000000
___register_frame_info .text 004018D0 00000026 0000001C 000000
Line 11 of 74

This particular line indicates that the _main function can be found in the .text section of the binary at virtual address 00401394, is (hex 08E) long, returns to the caller (R), and uses the EBP register (B) to reference its local variables.

Functions

Function name	Segment	Start	Length	Locals	Arguments	R	F	L	M	O	S	B	T	=
_gnu_exception_handler(x)	.text	00401000	0000013F	0000001C	00000004	R	.	.	.	.	.	B	T	.
___mingw_CRTStartup	.text	00401140	00000117	0000002C		.	.	.	.	.	.	B	.	.
_mainCRTStartup	.text	00401260	00000020	0000000C		.	.	L	.	.	.	B	.	.
_WinMainCRTStartup	.text	00401280	00000018	0000000C		.	.	.	.	.	.	B	.	.
_atexit	**.text**	**004012A0**	**0000000C**	**00000004**		**R**	**.**	**.**	**.**	**.**	**.**	**B**	**T**	**.**
__onexit	**.text**	**004012B0**	**0000000C**	**00000004**	**00000004**	**R**	**.**	**.**	**.**	**.**	**.**	**B**	**T**	**.**
___do_frame_init	.text	004012C0	00000022	0000000C		R	.	.	.	.	.	B	.	.
___do_frame_fini	.text	004012F0	00000014	0000000C		R	.	.	.	.	.	B	.	.
_shift	.text	00401310	0000005A	0000009C	00000004	R	.	.	.	.	.	B	T	.
_test	.text	0040136A	0000002A	0000000C	00000008	R	.	.	.	.	.	B	.	.
_main	**.text**	**00401394**	**0000008E**	**0000001C**	**0000000C**	**R**	**.**	**.**	**.**	**.**	**.**	**B**	**T**	**.**
__pei386_runtime_relocator	.text	00401430	00000028	00000004		R	.	.	.	.	.	B	.	.

As with other display windows, double-clicking an entry in the Functions window causes the disassembly window to jump to the location of the selected function.



The **Output window** serves as IDA's output console and is the place to look for information on tasks IDA is performing.

